Doc. No.: WG21/P0982R1
Date: 2018-11-06
Reply-to: Hans-J. Boehm
Email: hboehm@google.com
Authors: Hans-J. Boehm, Olivier Giroux, Viktor Vafeiades
Audience: WG21

P0982R1: Weaken Release Sequences

This proposal was forked from P0668R2. That paper contains three memory mode revision proposals, all of which are independent. SG1 decided that this piece of the memory model revision proposal was essentially ready to move forward, while other pieces require more discussion. It was suggested that this might be combined with P0735, which also deals with release sequences. But the proposals themselves, as well as the arguments pro and con, are entirely disjoint, so it seemed less confusing to keep them separate.

Release sequences were originally introduced to prevent relaxed read-modify-write operations, from breaking synchronizes-with relationships. If a thread initializes a data structure, and then, via a `memory_order_release` operation, sets an "initialized" bit in a word to signal that it has done so, other threads using e.g. `fetch_or` to set other bits should not interfere with that signaling mechanism. A thread observing the "initialized" bit via a `memory_order_acquire` operation is thus still guaranteed to see the data structure fully initialize.

If the "initialized" bit is set by a `memory_order_release operation`, additional bits in the same location are added using any atomic read-modify-write operations, and then the "initialized" bit is read via a `memory_order_acquire` load, we guarantee that the initial release operation still synchronizes with the final acquire load, even if the intervening read-modify-write operations are relaxed operations. In order for a release store to synchronize with an acquire load on the same location, the acquire load must observe either the value stored by the original release operation, *or another store operation in the "release sequence" headed by the release store*. Read-modify-write operations are included in the release sequence, and hence the appropriate synchronizes-with relationship is established, and a thread observing the "initialized" bit set is guaranteed to see the intialization of the associated object.

The standard implementation of reference counting relies heavily on the fact that atomic read-modify-write operations are included in release sequences.

Unfortunately, it was decided in the C++11 time frame that not only read-modify-write operations, but also `memory_order_relaxed` stores performed by the same thread that performed the original `memory_order_release` store, should be included in release sequences. This seemed reasonable at the time because this was expected to be a "free" property provided by existing architectures in any case. It was not motivated by good use cases at the time, and it is not clear there are any.

(There was a discussion of possible use cases on the reflector, starting on March 13, 2018. This suggested marginaly plausible scenarios in which such cases might possibly arise, but also reasons that such code remains extremely unlikely.)

It is becoming increasingly common to design hardware features (e.g. ARMv8 acquire loads and release stores) to match the C++ memory model specification. For such new designs, release sequences do impose additional constraints, as was pointed out by the hardware architects participating in the Toronto discussion of P0668. Thus weaker constraints are likely to benefit future hardware and thus C++ performance.

The problem with release sequences

This discussion largely follows Section 4.3 of Vafeiades et al, Common Compiler Optimisations are Invalid in the C11 Memory Model and what we can do about it, POPL 2015.

We can illustrate the problem with the example from the above paper:

Thread 1	Thread 2	Thread 3
$x =_{rlx} 2$	$y =_{na} 1$ $x =_{rel} 1$ $x =_{rlx} 3$	if (x_{acq} == 3) print(y)

Without Thread 1, this program is data-race-free. Thread 3 accesses `y` only if it sees a value of 3 for `x`, which must mean that it saw the second assignment to `x` by Thread 2. Since this assignment is performed by the same thread that performed the release store, with no intervening assignments to `x`, it is in the release sequence of the release store. Hence the second assignment synchronizes with the conditional in Thread 3. The program must print 1, if it prints anything at all.

Surprisingly, this no longer holds if we add Thread 1. If the Thread 1 assignment occurs (in `x`'s modification order) between the two assignments in Thread 2, then the release sequence is broken by this intervening assignment. There is no longer a synchronizes with relationship, and thus there is a data race.

This is highly counter-intuitive, since Thread 1 should have no impact on memory ordering in this case. This ability for other threads to interfere in such synchronizes-with relationships also makes it difficult to use this guarantee in correct code.

The existing definition also greatly complicates reasoning about C++ programs. Consider the above example with non-atomic operations replaced by relaxed operations. That program allows Thread 3 to print zero, since again the release sequence can be broken by Thread 1. However the program without Thread 1 does not allow this execution, in spite of the fact that no Thread actually observes the write by Thread 1. In all reasonable senses of the word, the program without Thread 1 is a prefix of the whole program. But the execution of the whole program is not an extension of the program prefix. This is problematic for both formal and informal reasoning about programs.

Relying on same thread writes in release sequences is also inherently brittle. Such use breaks if the final assignment is done by a helper thread instead of the original thread. For example, this does not interact well with the `task_block` `run` or `wait` functions from Parallelism TS v2, which may, mor or less implicitly, switch threads.

Proposed solution

P0668 suggested two different solutions to the core problem: Either we strengthen release sequences to prevent interference by other threads, or we weaken release sequences to no longer include same-thread-writes. The SG1 consensus in Jacksonville was in favor of the second solution, in spite of the fact that it is technically a breaking change. This was motivated largely by concerns about adding additional hardware constraints in support of a feature with no known use cases.

The proposal in P0668R2 was to deprecate same-thread writes in release sequences. The SG1 consensus in Jacksonville was to prefer outright removal. That's the wording change we present here.

Update Section 6.8.2.1p5 [intro.races] to read:

> A release sequence headed by a release operation *A* on an atomic object *M* is a maximal contiguous sub-sequence of side effects in the modification order of *M*, where the first operation is *A*, and every subsequent operation ~~is performed by the same thread that performed *A*, or~~ is an atomic read-modify-write operation.

(The list formatting was also removed.)

Add a new section in Clause C.5.*N* (position and *N* determined by the editor):

> **C.5.*N* Same-thread stores in release sequences**
>
> Affected subclause: 6.8.2.1 [intro.races]
>
> Change: Except for the initial release operation, a release sequence consists solely of atomic read-modify-write operations.
>
> Rationale: Removal of rarely used and confusing feature.
>
> Effect on original feature: If a `memory_order_release` atomic store is followed by a `memory_order_relaxed` store to the same variable by the same thread, then reading the latter value with a `memory_order_acquire` load no longer provides any "happens before" guarantees, even in the absence of intervening stores by another thread.